# DO CONTEXT ENGINES HELP? RETRIEVAL QUALITY, DETERMINISM, AND AGENT UTILITY UNDER MATCHED BASELINES AND AUDITED EXPOSURE

**Anonymous authors**
Paper under double-blind review

## ABSTRACT

Coding agents are increasingly wrapped in "context engines" that index a repository and return the files the model might need. We evaluate one open-source engine, Delphi, against lexical baselines, a matched ladder of conventional retrievers, and two hosted engines, under a protocol that records at the case level what each evaluation set could have influenced. Three findings temper our own earlier claims. *Exposure:* the largest partition we had called "untouched"—220 Agent Retrieval Bench cases—was re-drawn from a pool whose cases had all been scored in an earlier round and whose aggregate results gated which retrieval components shipped; we relabel it validation evidence and rest confirmatory claims on sets never scored before their single run: 62 SWE-bench Verified localization instances, 18 repository-disjoint commit-to-files cases, and a further 98 Verified instances drawn under a fresh salt. *Baselines:* Delphi beats BM25 and the official lexical ranker on every set, but against a conventional dense+BM25 hybrid built from the same embedding model, with Delphi's own cross-encoder, listwise reranker, and query expansion attached, the picture splits: on 160 pooled SWE-bench Verified localization instances Delphi leads that stack by $+0.06$–$0.08$ on Recall@5, Recall@20, and BCY with repository-cluster intervals excluding zero and ties on MRR ($+0.057$ $[-0.036, +0.114]$); on the 18-case independent final the conventional stack leads with intervals excluding zero. The reranking head, not anything specific to Delphi, accounts for most of the margin over lexical systems ($+0.3$ MRR versus $+0.06$). *Contracts:* routing every engine's retrieved evidence, including the hosted synthesis engine's own retrieved sources, through one frozen synthesis stage yields a four-way documentation tie ($0.625/0.617/0.583/0.575$) sitting $0.08$–$0.13$ above a $0.492$ no-retrieval floor; no engine-versus-engine interval excludes zero. On like-for-like retrieved-set stability the hosted synthesis engine is fully stable and Delphi is $98\%$ stable; the "$0\%$" we had reported for it measured generated text. A preregistered executable pilot on the same 62 SWE-bench instances (mini-SWE-agent, fixed budget, two trajectories per cell) resolves no seed effect: a Delphi seed changes verified repair by $+0.048$ $[-0.008, +0.113]$ over no seed and by $-0.008$ over a random-file control, and the two repeats disagree by more than the effects under study. We claim no state of the art. The contribution is the protocol, the negative and null results, and an exposure ledger that lets a reader check which number can bear which claim.

## 1 INTRODUCTION

The pitch for a context engine is simple. Index the repo. Embed the chunks. When the agent asks a question, return the files that matter. The pitch for *not* building one is simpler: `rg` is deterministic, takes 30 milliseconds, and already found `src/_pytest/assertion/rewrite.py` when we asked where pytest rewrites asserts.

We built Delphi as a local-first retrieval stack—hybrid lexical/vector search, query expansion, a cross-encoder, a listwise rerank—and spent two evaluation rounds measuring it. The first round reported large wins over lexical baselines on a partition we called untouched. External review asked three questions we could not answer from the draft: what information from the earlier round was available while the shipped system was chosen; how much of the margin survives against a conventional dense-plus-lexical hybrid that uses the same reranker; and whether any of it changes what a coding agent actually repairs. This paper is the answer, and much of it is unflattering.

1. **RQ1 (utility).** Holding the agent, its tools, and its budget fixed, does a retrieval seed change what it submits—and does it change verified repair?
2. **RQ2 (ranking).** Against lexical baselines, against a matched conventional hybrid, and against hosted engines, where does Delphi lead, tie, or trail, on cases that no development decision could see?
3. **RQ3 (determinism).** Where does run-to-run variance come from, and when the same quantity is measured for every engine, who is actually stable?

Short answers. **RQ2:** Delphi leads lexical baselines everywhere; against a conventional ladder built from its own parts it leads modestly on 160 SWE-bench issue queries (three of four metrics excluding zero) and trails on the 18-case commit-to-files final (§4). **RQ3:** the variance was ours and is fixed in-process, but the like-for-like comparison shows the hosted synthesis engine's retrieval is at least as stable as ours (§5). **RQ1:** seeds move a closed-tool agent's file selection, replicating the benchmark's own intervention; on executable repair no seed effect is resolved at $n$=62 with two trajectories, and a random-file block does as well as Delphi's (§7).

## 2 Related work

SWE-bench established executable repository-level issue resolution (Jimenez et al., 2024), while SWE-agent and Agentless expose repository navigation and localization as explicit stages (Yang et al., 2024; Xia et al., 2024). LocAgent studies a graph-guided localization agent and reports that better localization can improve multi-attempt repair (Chen et al., 2025). Repository-level code completion provides complementary evidence that cross-file retrieval matters once the edit location is known: RepoCoder alternates retrieval and generation (Zhang et al., 2023); CodeRAG adds multi-path retrieval and preference-aligned reranking (Zhang et al., 2025). Dense retrieval, late interaction, and hypothetical-document expansion motivate the components in practical context engines (Karpukhin et al., 2020; Khattab & Zaharia, 2020; Gao et al., 2023), while BM25 remains a strong exact-term baseline for code (Robertson & Zaragoza, 2009).

The closest work measures context acquisition directly. ContextBench provides human-verified file, block, and line contexts for issue-resolution tasks and scores agent trajectories (Li et al., 2026). CORE-Bench scales requirement-driven retrieval across code understanding, edit localization, and broader context (Zhang et al., 2026a). SWE-Explore derives audited line-level targets from successful trajectories (Zhang et al., 2026b). Agent Retrieval Bench (ARB) contributes workflow-derived next-context targets, base-commit hygiene, budget-aware file metrics, open-embedding and RepoMap baselines, and a closed-tool seed intervention with no-seed, random, retrieval-derived, and oracle arms (Qin & Xie, 2026). Our closed-tool experiment (§7) is a replication of that intervention with an additional seed, not a new design. What we add to this line is narrower: an exposure ledger that classifies every case by what could have influenced it, a component-matched baseline ladder, matched output contracts for hosted engines, like-for-like determinism, and a preregistered executable pilot on the same tasks as the ranking evaluation.

## 3 Protocol

**Exposure classes.** Every evaluation set is assigned a class in a case-level ledger (Appendix B). **C0**: never scored by any system before its single confirmatory run, drawn from a pool no development decision could see. **C1**: development. **C2**: re-partitioned from a pool whose cases were all scored in an earlier round, where that round's aggregates influenced the shipped system. The ARB round-3 "final" is C2: all 427 ARB v2 cases were scored in July 2026; the July held-out aggregates were the acceptance test for query expansion, listwise reranking, the cross-encoder blend, reciprocal-rank fusion, and the path-affinity branch, and the July per-workflow rates motivated the quoted-path demotion fix. Round 3 then drew a new $25/75$ split from the same pool, excluded 50 legacy-exposed IDs, and queried the 220-case partition once. Re-drawing does not un-expose a case. Round-2 per-case outputs are not retained, so per-case tuning can neither be ruled in nor out from the archive; we therefore treat the partition as validation evidence throughout and make no confirmatory claim on it. The C0 sets are: 62 stratified SWE-bench Verified localization instances (12 repositories; patch-marker leakage audit $0/62$); an 18-case commit-to-files final over a repository-disjoint corpus locked before any scoring; and 100 further Verified instances drawn on 2026-09-09 under a fresh salt from the 438 instances no round had used. A 40-case DS-1000 development subset drives the documentation track and is C1. The ledger controls author exposure; it cannot control whether the foundation models inside any engine saw these public repositories during training, a caveat shared by every arm.

**Queries, corpus, metrics.** Queries are the official ARB gold-free structured objects, serialized with sorted keys. File text comes from bare git clones at each case's `base_commit` (materialized snapshots; binary and non-UTF-8 blobs skipped). Metrics are the official ARB `sample_metrics` (MRR, Recall@$k$) plus BCY@8k, the budget-constrained yield from packing ranked files into an 8,000-token context. Documentation cases score *identifier hit*: whether the returned text contains the case's gold API identifier.

**Freeze and attestation.** All Delphi numbers come from one frozen commit (`91d76c1`) with exact vector scan, API top-$k$=20, and response-level serving-configuration attestation: every response carries the retrieval configuration actually served, and a run aborts on the first mismatch with its declared configuration. The frozen configuration is reproduced in full in Appendix A. Paired deltas carry cluster bootstrap $95\%$ intervals (20,000 resamples, seed 1042; repository clusters for repository tracks, case clusters for documentation), plus exact McNemar tests on any-gold@20 movement.

**Matched baseline ladder.** The lexical comparators (whole-file BM25, ARB's official lexical ranker, and their development-tuned fusion) cannot say how much of a margin is Delphi's engineering and how much is a learned reranker on top of anything. We therefore add four conventional systems, each adding one Delphi component to the previous one, over the official ARB chunks (file chunk plus symbol chunks) of the identical corpus: **dense**, `text-embedding-3-small`—the embedding model inside Delphi—max-pooled to files; **hybrid**, reciprocal-rank fusion ($k$=60, equal weights, untuned) of the dense ranking and BM25; **+ rerankers**, the hybrid's top 50 narrowed to 30 through Delphi's own cross-encoder (`ms-marco-MiniLM-L-6-v2`, blend $0.4$) and then Delphi's own listwise `gpt-4o` reranker over the top 20 (seed 1042, identical prompt); **+ expansion**, the same with Delphi's hypothetical-document expansion (`gpt-4o-mini`, identical prompt and gate) feeding the dense query. The prompts and parsers are imported from Delphi's source so they are identical by construction. What remains between the last rung and Delphi is exactly Delphi's own contribution: exact-symbol, exact-path, path-affinity, and trigram branches; its chunker and index; tuned fusion weights; quoted-path demotion. Reported latencies exclude index construction for every system, as they did for Delphi.

**Claim rule.** "State of the art" would require leading every directly comparable engine on a C0 set with the complete comparable engine set runnable. No result in this paper meets it.

## 4 RQ2: REPOSITORY RETRIEVAL

**Against lexical baselines.** Delphi leads whole-file BM25, the official lexical ranker, and their tuned fusion on every set. On the independent final the Recall@20 leads over BM25 ($+0.074$ [$+0.019$, $+0.139$]) and the lexical ranker ($+0.046$ [$+0.009$, $+0.083$]) exclude zero while the fusion comparison is a tie; on SWE-bench MRR leads the strongest lexical system by $+0.274$ [$+0.194$, $+0.412$] with Recall@20 tied; on the C2 ARB partition every lexical comparison excludes zero. None of this is in dispute, and none of it isolates Delphi's contribution.

**Against the matched ladder.** Table 1 is the first of the two clean tests. The dense rung alone, using nothing but the embedding model Delphi already calls, ties Delphi on any-gold and exceeds it on Re-

Table 1: Independent commit-to-files final (C0): 18 cases, 6 repositories, scored once. Any-gold is the number of cases with a gold file in the top 20; latency is the median query time in seconds. $\Delta$ rows are Delphi minus the named system.

| System | MRR | R@5 | R@20 | BCY@8k | any-gold | s |
|---|---|---|---|---|---|---|
| **Delphi (frozen)** | **0.508** | **0.551** | **0.750** | **0.454** | 15/18 | 3.7 |
| Dense (same embedding) | 0.455 | 0.523 | 0.796 | 0.347 | 15/18 | 0.5 |
| Dense+BM25 RRF | 0.579 | 0.639 | 0.870 | 0.403 | 16/18 | 0.2 |
| + Delphi's rerankers | 0.545 | 0.644 | 0.870 | 0.454 | 16/18 | 2.3 |
| + expansion | 0.619 | 0.662 | 0.852 | 0.546 | 16/18 | 4.1 |
| Lexical+BM25 RRF | 0.473 | 0.500 | 0.759 | 0.287 | 15/18 | 1.6 |
| Lexical ranker | 0.438 | 0.403 | 0.704 | 0.259 | 14/18 | 1.1 |
| BM25 | 0.370 | 0.458 | 0.676 | 0.259 | 14/18 | 1.2 |
| *Paired Delphi−system deltas, repository-cluster bootstrap 95% intervals; * interval excludes zero* | | | | | | |
| $\Delta$ vs. Dense (same embedding) | $+0.053$ $[-0.084, +0.190]$ | $+0.028$ $[-0.148, +0.194]$ | $-0.046$ $[-0.139, +0.000]$ | $+0.106$ $[-0.074, +0.259]$ | 15 vs 15 | $p$=1 |
| $\Delta$ vs. Dense+BM25 RRF | $-0.070$ $[-0.222, +0.086]$ | $-0.088$ $[-0.236, +0.056]$ | $-0.120^{*}$ $[-0.231, -0.019]$ | $+0.051$ $[-0.120, +0.222]$ | 15 vs 16 | $p$=1 |
| $\Delta$ vs. + Delphi's rerankers | $-0.037$ $[-0.123, +0.049]$ | $-0.093$ $[-0.324, +0.083]$ | $-0.120^{*}$ $[-0.231, -0.019]$ | $+0.000$ $[-0.139, +0.194]$ | 15 vs 16 | $p$=1 |
| $\Delta$ vs. + expansion | $-0.111^{*}$ $[-0.200, -0.034]$ | $-0.111$ $[-0.324, +0.046]$ | $-0.102$ $[-0.213, +0.000]$ | $-0.093$ $[-0.278, +0.074]$ | 15 vs 16 | $p$=1 |
| $\Delta$ vs. Lexical+BM25 RRF | $+0.035$ $[-0.211, +0.259]$ | $+0.051$ $[-0.088, +0.194]$ | $-0.009$ $[-0.148, +0.083]$ | $+0.167$ $[-0.074, +0.472]$ | 15 vs 15 | $p$=1 |
| $\Delta$ vs. Lexical ranker | $+0.070$ $[-0.177, +0.276]$ | $+0.148$ $[-0.046, +0.343]$ | $+0.046^{*}$ $[+0.009, +0.083]$ | $+0.194$ $[-0.046, +0.481]$ | 15 vs 14 | $p$=1 |
| $\Delta$ vs. BM25 | $+0.138$ $[-0.067, +0.301]$ | $+0.093$ $[-0.037, +0.278]$ | $+0.074^{*}$ $[+0.019, +0.139]$ | $+0.194$ $[-0.046, +0.481]$ | 15 vs 14 | $p$=1 |

Table 2: SWE-bench Verified localization (C0): 62 instances, 12 repositories, scored once. Columns as in Table 1.

| System | MRR | R@5 | R@20 | BCY@8k | any-gold | s |
|---|---|---|---|---|---|---|
| **Delphi (frozen)** | **0.680** | **0.704** | **0.737** | **0.638** | 48/62 | 3.6 |
| Dense (same embedding) | 0.254 | 0.324 | 0.591 | 0.210 | 39/62 | 0.5 |
| Dense+BM25 RRF | 0.296 | 0.452 | 0.648 | 0.234 | 43/62 | 2.0 |
| + Delphi's rerankers | 0.575 | 0.598 | 0.616 | 0.533 | 41/62 | 3.7 |
| + expansion | 0.597 | 0.623 | 0.657 | 0.573 | 44/62 | 6.2 |
| Lexical+BM25 RRF | 0.354 | 0.500 | 0.719 | 0.306 | 47/62 | 13.2 |
| Lexical ranker | 0.406 | 0.509 | 0.702 | 0.387 | 46/62 | 11.0 |
| BM25 | 0.149 | 0.230 | 0.416 | 0.145 | 29/62 | 10.7 |
| *Paired Delphi−system deltas, repository-cluster bootstrap 95% intervals; * interval excludes zero* | | | | | | |
| $\Delta$ vs. Dense (same embedding) | $+0.426^{*}$ $[+0.328, +0.492]$ | $+0.380^{*}$ $[+0.262, +0.583]$ | $+0.146$ $[+0.000, +0.288]$ | $+0.427^{*}$ $[+0.340, +0.500]$ | 48 vs 39 | $p$=0.064 |
| $\Delta$ vs. Dense+BM25 RRF | $+0.384^{*}$ $[+0.275, +0.464]$ | $+0.252^{*}$ $[+0.152, +0.442]$ | $+0.089$ $[-0.038, +0.182]$ | $+0.404^{*}$ $[+0.288, +0.517]$ | 48 vs 43 | $p$=0.3 |
| $\Delta$ vs. + Delphi's rerankers | $+0.104$ $[-0.065, +0.210]$ | $+0.106$ $[+0.000, +0.263]$ | $+0.121$ $[-0.033, +0.259]$ | $+0.105$ $[+0.000, +0.190]$ | 48 vs 41 | $p$=0.14 |
| $\Delta$ vs. + expansion | $+0.083$ $[-0.107, +0.203]$ | $+0.081$ $[-0.056, +0.251]$ | $+0.080$ $[-0.105, +0.217]$ | $+0.065$ $[-0.062, +0.171]$ | 48 vs 44 | $p$=0.42 |
| $\Delta$ vs. Lexical+BM25 RRF | $+0.325^{*}$ $[+0.216, +0.453]$ | $+0.204^{*}$ $[+0.099, +0.455]$ | $+0.018$ $[-0.073, +0.075]$ | $+0.331^{*}$ $[+0.211, +0.552]$ | 48 vs 47 | $p$=1 |
| $\Delta$ vs. Lexical ranker | $+0.274^{*}$ $[+0.194, +0.412]$ | $+0.195^{*}$ $[+0.107, +0.403]$ | $+0.035$ $[-0.042, +0.111]$ | $+0.251^{*}$ $[+0.157, +0.433]$ | 48 vs 46 | $p$=0.75 |
| $\Delta$ vs. BM25 | $+0.530^{*}$ $[+0.450, +0.662]$ | $+0.474^{*}$ $[+0.352, +0.726]$ | $+0.321^{*}$ $[+0.189, +0.572]$ | $+0.493^{*}$ $[+0.395, +0.669]$ | 48 vs 29 | $p$=0.00055 |

call@20 (0.796 vs. 0.750). Adding BM25 by untuned reciprocal-rank fusion yields MRR 0.579 and Recall@20 0.870 against Delphi's 0.508 and 0.750; the Recall@20 gap ($-0.120$ $[-0.231, -0.019]$) excludes zero. Attaching Delphi's own rerankers and expansion reaches MRR 0.619 and BCY 0.546, and Delphi's MRR deficit against that rung ($-0.111$ $[-0.200, -0.034]$) also excludes zero. Six repository clusters is a small basis for an interval and we report the direction with that caveat; but the point is not that the conventional stack wins, it is that nothing Delphi adds on top of it is visible here.

Table 2 is the second clean test. Its point estimates lean the other way and its conclusion is the same. On issue text the dense rung alone is weak (MRR 0.254) and untuned fusion with BM25 helps little (0.296); Delphi's leads over both have intervals excluding zero. Attaching Delphi's rerankers to that hybrid moves MRR from 0.296 to 0.575, and expansion to 0.597: the reranking head, not the candidate branches, is where the margin over lexical systems comes from. Against the final rung Delphi leads by $+0.083$ MRR $[-0.107, +0.203]$, $+0.080$ Recall@20 $[-0.105, +0.217]$, and $+0.065$ BCY $[-0.062, +0.171]$, with any-gold 48 vs. 44 ($p$=0.42)—a tie. Across the two C0 sets the sign of Delphi's difference from the conventional stack flips, neither set resolves it in Delphi's favor, and the only resolved differences (independent Recall@20 and MRR) favor the conventional stack. On the C2 ARB partition (Table 3) Delphi's margin over the final rung is $+0.103$ MRR $[-0.008, +0.189]$ and $+0.076$ BCY $[-0.019, +0.157]$, with Recall@5 and Recall@20 tied and any-gold 163 vs. 148 ($p$=0.08); against every earlier rung the leads exclude zero. That is the set on which Delphi's components were selected, so a margin there is the expected direction of selection, not evidence. Its workflow breakdown is still informative as a hypothesis: Delphi's lead is concentrated in `trace2code` (MRR 0.579 vs. 0.149; Recall@20 0.895 vs. 0.368) and `edit2ripple`, whose queries carry file paths and symbol names that its exact-path, path-affinity, and symbol branches match directly, while on `code2test` and `comment2context` the conventional rung is equal or ahead. An exploratory split of the 62 SWE-bench issues by whether they contain a traceback or a

Table 3: ARB round-3 partition (C2, validation evidence only): 220 cases, 19 repositories. Columns as in Table 1. This table is reported for completeness; no confirmatory claim rests on it.

| System | MRR | R@5 | R@20 | BCY@8k | any-gold | s |
|---|---|---|---|---|---|---|
| **Delphi (frozen)** | **0.332** | **0.476** | **0.648** | **0.296** | 163/220 | 4.3 |
| Dense (same embedding) | 0.168 | 0.218 | 0.458 | 0.118 | 115/220 | 0.5 |
| Dense+BM25 RRF | 0.195 | 0.247 | 0.533 | 0.141 | 137/220 | 0.5 |
| + Delphi's rerankers | 0.214 | 0.317 | 0.526 | 0.176 | 135/220 | 2.1 |
| + expansion | 0.229 | 0.389 | 0.595 | 0.220 | 148/220 | 4.2 |
| Lexical+BM25 RRF | 0.160 | 0.204 | 0.518 | 0.124 | 131/220 | 1.9 |
| Lexical ranker | 0.144 | 0.215 | 0.478 | 0.138 | 123/220 | 1.4 |
| BM25 | 0.128 | 0.177 | 0.425 | 0.075 | 113/220 | 1.3 |
| *Paired Delphi−system deltas, repository-cluster bootstrap 95% intervals; * interval excludes zero* | | | | | | |
| $\Delta$ vs. Dense (same embedding) | $+0.164^*$ [$+0.049$, $+0.257$] | $+0.258^*$ [$+0.090$, $+0.401$] | $+0.190^*$ [$+0.029$, $+0.321$] | $+0.178^*$ [$+0.058$, $+0.286$] | 163 vs 115 | $p$=5.9$e-08$ |
| $\Delta$ vs. Dense+BM25 RRF | $+0.137^*$ [$+0.036$, $+0.215$] | $+0.229^*$ [$+0.074$, $+0.366$] | $+0.115$ [$-0.039$, $+0.237$] | $+0.155^*$ [$+0.060$, $+0.246$] | 163 vs 137 | $p$=0.0022 |
| $\Delta$ vs. + Delphi's rerankers | $+0.118^*$ [$+0.009$, $+0.198$] | $+0.159^*$ [$+0.007$, $+0.286$] | $+0.122$ [$-0.031$, $+0.240$] | $+0.120^*$ [$+0.039$, $+0.203$] | 163 vs 135 | $p$=0.0011 |
| $\Delta$ vs. + expansion | $+0.103$ [$-0.008$, $+0.189$] | $+0.087$ [$-0.100$, $+0.238$] | $+0.053$ [$-0.117$, $+0.195$] | $+0.076$ [$-0.019$, $+0.157$] | 163 vs 148 | $p$=0.082 |
| $\Delta$ vs. Lexical+BM25 RRF | $+0.172^*$ [$+0.083$, $+0.235$] | $+0.272^*$ [$+0.148$, $+0.372$] | $+0.130^*$ [$+0.024$, $+0.200$] | $+0.172^*$ [$+0.098$, $+0.263$] | 163 vs 131 | $p$=0.00017 |
| $\Delta$ vs. Lexical ranker | $+0.188^*$ [$+0.106$, $+0.246$] | $+0.261^*$ [$+0.140$, $+0.361$] | $+0.170^*$ [$+0.088$, $+0.226$] | $+0.158^*$ [$+0.076$, $+0.255$] | 163 vs 123 | $p$=2.4$e-06$ |
| $\Delta$ vs. BM25 | $+0.204^*$ [$+0.111$, $+0.277$] | $+0.299^*$ [$+0.157$, $+0.433$] | $+0.223^*$ [$+0.111$, $+0.310$] | $+0.221^*$ [$+0.129$, $+0.303$] | 163 vs 113 | $p$=5$e-09$ |

Table 4: SWE-bench Verified expansion (C0): 98 further instances drawn under a fresh salt and never used in any round, 11 repositories, scored once from the frozen commit after a complete index audit (182,415 files; 1,079,743 chunks and embeddings; zero mismatches) and the gold-searchability preflight. Columns as in Table 1.

| System | MRR | R@5 | R@20 | BCY@8k | any-gold | s |
|---|---|---|---|---|---|---|
| **Delphi (frozen)** | **0.700** | **0.745** | **0.820** | **0.685** | 83/98 | 6.4 |
| Dense (same embedding) | 0.214 | 0.389 | 0.673 | 0.156 | 69/98 | 0.8 |
| Dense+BM25 RRF | 0.306 | 0.509 | 0.651 | 0.223 | 67/98 | 2.0 |
| + Delphi's rerankers | 0.611 | 0.622 | 0.667 | 0.576 | 69/98 | 3.8 |
| + expansion | 0.660 | 0.684 | 0.733 | 0.603 | 76/98 | 6.2 |
| Lexical+BM25 RRF | 0.383 | 0.521 | 0.789 | 0.350 | 81/98 | 4.2 |
| Lexical ranker | 0.487 | 0.603 | 0.793 | 0.445 | 82/98 | 2.2 |
| BM25 | 0.168 | 0.238 | 0.457 | 0.146 | 48/98 | 2.2 |
| *Paired Delphi−system deltas, repository-cluster bootstrap 95% intervals; * interval excludes zero* | | | | | | |
| $\Delta$ vs. Dense (same embedding) | $+0.486^*$ [$+0.434$, $+0.622$] | $+0.356^*$ [$+0.281$, $+0.496$] | $+0.147^*$ [$+0.083$, $+0.330$] | $+0.529^*$ [$+0.453$, $+0.656$] | 83 vs 69 | $p$=0.0043 |
| $\Delta$ vs. Dense+BM25 RRF | $+0.395^*$ [$+0.299$, $+0.552$] | $+0.236^*$ [$+0.133$, $+0.513$] | $+0.168^*$ [$+0.116$, $+0.307$] | $+0.463^*$ [$+0.391$, $+0.636$] | 83 vs 67 | $p$=0.0015 |
| $\Delta$ vs. + Delphi's rerankers | $+0.089^*$ [$+0.022$, $+0.227$] | $+0.123^*$ [$+0.067$, $+0.277$] | $+0.153^*$ [$+0.099$, $+0.298$] | $+0.110^*$ [$+0.062$, $+0.238$] | 83 vs 69 | $p$=0.0043 |
| $\Delta$ vs. + expansion | $+0.041$ [$-0.028$, $+0.112$] | $+0.061^*$ [$+0.023$, $+0.148$] | $+0.087^*$ [$+0.042$, $+0.167$] | $+0.082^*$ [$+0.030$, $+0.152$] | 83 vs 76 | $p$=0.17 |
| $\Delta$ vs. Lexical+BM25 RRF | $+0.317^*$ [$+0.229$, $+0.466$] | $+0.224^*$ [$+0.141$, $+0.415$] | $+0.031$ [$-0.045$, $+0.205$] | $+0.335^*$ [$+0.247$, $+0.472$] | 83 vs 81 | $p$=0.79 |
| $\Delta$ vs. Lexical ranker | $+0.213^*$ [$+0.135$, $+0.404$] | $+0.142^*$ [$+0.050$, $+0.396$] | $+0.027$ [$-0.035$, $+0.163$] | $+0.241^*$ [$+0.150$, $+0.439$] | 83 vs 82 | $p$=1 |
| $\Delta$ vs. BM25 | $+0.533^*$ [$+0.453$, $+0.668$] | $+0.507^*$ [$+0.405$, $+0.750$] | $+0.362^*$ [$+0.279$, $+0.554$] | $+0.539^*$ [$+0.455$, $+0.712$] | 83 vs 48 | $p$=5.5$e-10$ |

Python path does not reproduce that pattern (Delphi's largest advantage is on issues with neither, $n$=35), so we record it as a hypothesis for a designed test, not a finding.

**The expansion, and the pooled picture.** To widen the C0 basis we drew 100 further Verified instances under a fresh salt from the 438 no round had used, applied two engine-agnostic rules before any scoring (a gold file must exist at the base commit, which removed one patch-created file from one instance's gold set; ARB's leakage check, which excluded two instances whose issue text carried patch markers), provisioned the frozen stack on the 98 remaining snapshots, and scored every system once (Table 4). The shape repeats: dense and untuned hybrid are weak on issue text, the reranking head does most of the work ($0.306 \to 0.611$ MRR), expansion adds a little (0.660), and Delphi finishes at 0.700. On this larger set Delphi's margin over the final rung is $+0.041$ MRR [$-0.028$, $+0.112$], $+0.061$ Recall@5 [$+0.023$, $+0.148$], $+0.087$ Recall@20 [$+0.042$, $+0.167$], and $+0.082$ BCY [$+0.030$, $+0.152$]: three of four intervals exclude zero. Pooling the 160 SWE-bench instances (Table 5) gives the same reading, $+0.057$ MRR [$-0.036$, $+0.114$] with Recall@5, Recall@20, and BCY all excluding zero and any-gold 131 vs. 120 ($p$=0.08). Against the lexical ranker, which is strong at Recall@20 on issue text (0.793), Delphi's pooled Recall@20 lead is a tie ($+0.030$ [$-0.019$, $+0.117$]) while its MRR and BCY leads exclude zero.

So the answer to the reviewers' question is not one number. On SWE-bench-style issue queries Delphi's own additions are worth roughly $+0.06$ to $+0.09$ on the yield metrics over a conventional stack built from its own parts, resolved at $n$=160; on the commit-to-files set they are worth less than nothing, resolved at $n$=18. In both cases the component that matters most is the one any practitioner would add first: a reranking head over a dense-plus-lexical candidate pool, which moves MRR by $+0.3$ where Delphi's remaining engineering moves it by $+0.06$.

**Development, briefly.** The development partitions drove preregistered ablations whose outcomes we keep regardless of direction: exact vector scan replaced HNSW ($+0.020$ Recall@20

Table 5: Pooled SWE-bench Verified localization, 160 C0 instances (Tables 2 and 4), 12 repositories. Paired Delphi$-$system deltas with repository-cluster bootstrap 95% intervals; $^*$ excludes zero.

| Delphi $-$ system | MRR | R@5 | R@20 | BCY@8k | any-gold ($p$) |
|---|---|---|---|---|---|
| Dense (same embedding) | $+0.463^*$ [$+0.418$, $+0.547$] | $+0.365^*$ [$+0.296$, $+0.505$] | $+0.147^*$ [$+0.086$, $+0.277$] | $+0.490^*$ [$+0.434$, $+0.568$] | 131 vs 108 ($p$=0.00043) |
| Dense+BM25 RRF | $+0.390^*$ [$+0.316$, $+0.492$] | $+0.242^*$ [$+0.154$, $+0.470$] | $+0.138^*$ [$+0.090$, $+0.226$] | $+0.440^*$ [$+0.394$, $+0.546$] | 131 vs 110 ($p$=0.0011) |
| + Delphi's rerankers | $+0.095^*$ [$+0.023$, $+0.181$] | $+0.116^*$ [$+0.064$, $+0.242$] | $+0.141^*$ [$+0.084$, $+0.248$] | $+0.108^*$ [$+0.064$, $+0.188$] | 131 vs 110 ($p$=0.0011) |
| + expansion | $+0.057$ [$-0.036$, $+0.114$] | $+0.069^*$ [$+0.015$, $+0.160$] | $+0.084^*$ [$+0.013$, $+0.154$] | $+0.076^*$ [$+0.017$, $+0.128$] | 131 vs 120 ($p$=0.08) |
| Lexical+BM25 RRF | $+0.320^*$ [$+0.255$, $+0.434$] | $+0.216^*$ [$+0.151$, $+0.393$] | $+0.026$ [$-0.031$, $+0.128$] | $+0.334^*$ [$+0.286$, $+0.453$] | 131 vs 128 ($p$=0.69) |
| Lexical ranker | $+0.237^*$ [$+0.174$, $+0.391$] | $+0.163^*$ [$+0.081$, $+0.368$] | $+0.030$ [$-0.019$, $+0.117$] | $+0.245^*$ [$+0.180$, $+0.406$] | 131 vs 128 ($p$=0.66) |
| BM25 | $+0.532^*$ [$+0.491$, $+0.630$] | $+0.494^*$ [$+0.411$, $+0.738$] | $+0.346^*$ [$+0.263$, $+0.538$] | $+0.521^*$ [$+0.478$, $+0.645$] | 131 vs 77 ($p$=$2.7e-12$) |

Table 6: Like-for-like repeatability on ten documentation cases $\times$ ten runs (450 run pairs per engine). Retrieved-set columns use each engine's observable retrieval unit: retrieved items for Delphi and the documentation engine, the set of cited URLs for the synthesis engine. Byte-exact is the delivered context text; hit agreement is whether the identifier-hit outcome agreed across the pair.

| Engine | Set exact | Order exact | Set Jaccard | Byte exact | Hit agreement |
|---|---|---|---|---|---|
| Delphi (raw context, $k$=5) | 0.980 | 0.980 | 0.993 | 0.980 | 1.000 |
| Documentation engine (quality-guided IDs) | 0.542 | 0.362 | 0.835 | 0.356 | 0.918 |
| Documentation engine (oracle IDs) | 0.496 | 0.258 | 0.806 | 0.253 | 0.909 |
| Synthesis engine (cited-URL set) | 1.000 | 1.000 | 1.000 | 0.000 | 0.929 |

[0.000, 0.063]); query expansion stayed on; a Gemini embedding swap was rejected (MRR $-0.054$ [$-0.102$, $-0.011$]); and a file-level Okapi BM25 branch was rejected when its single approved configuration failed preregistered gates (MRR interval floor $-0.050$, BCY floor $-0.037$, median latency ratio 1.34) despite losing zero any-gold cases. It ships default-off. Two development findings moved code: review-comment queries retrieved the file the comment already quoted, so paths present in the query envelope are demoted after the cross-encoder; and a global `*.pb.go` exclusion had silently dropped a gold file, so agent-mode indexing now retains generated source under the existing size and binary guards. Both were selected on development or C2 evidence, which is exactly why they cannot be credited from Table 3.

## 5 RQ3: DETERMINISM, LIKE FOR LIKE

**The pipeline was the variance.** Repeating eight ARB queries ten times against the July stack gave a mean exact top-10 rate of 30.6%. Paired traces isolated uncached, unseeded LLM stages; insertion-order ties in fusion SQL; and approximate HNSW under concurrency. Total SQL ordering, single-flight sharing of concurrent identical hosted calls, an explicit seed, and a persistent stage cache take exact repeats to 100% in-process across two concurrent $8\times10$ packets and two 75-case repeats; a genuine cold restart still moved $7/8$ lists until the cache was warm, after which $8/8$ survive two restarts. We claim restart-durable repeatability from the first persisted answer, not that two clean installations agree on their first answer. Embedding APIs were not the problem: the small OpenAI model jitters at the fifth decimal and never changed top-ten membership across $N$=20 repeats; Gemini was bitwise deterministic.

**Same quantity, every engine.** Our earlier headline—98.0% byte-identical context for Delphi against 35.6% and 0.0% for the hosted engines—compared retrieval stability for two engines with generated-text stability for the third. Table 6 separates the layers. On retrieved-set stability the synthesis engine's cited sources are identical across every pair (1.000), Delphi's retrieved items across 98.0%, and the documentation engine's across 54% (quality-guided) or 50% (oracle IDs). What differs for the synthesis engine is the layer above retrieval: its answer text never repeats byte-for-byte, and that variation flips the identifier-hit outcome in 7.1% of pairs, against 0% for Delphi. The defensible statements are narrower than before: Delphi's caching and total ordering make its retrieved text repeatable, and repeatable retrieved text makes downstream decisions repeatable; the documentation engine's retrieval drifts; the synthesis engine's retrieval does not drift, its prose does. We have not tested whether a caching wrapper would give the hosted engines the same repeatability, and expect that it would.

Table 7: Matched-output-contract documentation comparison, 40 development cases (C1) $\times$ 3 repeats through one frozen synthesis stage. Deltas are case-cluster bootstrap 95% intervals against the no-retrieval control. The synthesis engine's native single pass is shown for reference only.

| Arm | Identifier hit | Per-repeat | $\Delta$ vs. control |
|---|---|---|---|
| Documentation engine retrieval + frozen synthesis | 0.625 | 0.650/0.600/0.625 | +0.133 [+0.008, +0.258] |
| **Delphi retrieval + frozen synthesis** | 0.617 | 0.600/0.625/0.625 | +0.125 [+0.000, +0.250] |
| Synthesis engine retrieval + frozen synthesis | 0.583 | 0.625/0.525/0.600 | +0.092 [−0.008, +0.200] |
| Synthesis engine, native synthesis (recorded) | 0.575 | single pass | — |
| Model alone (no retrieval) | 0.492 | 0.475/0.475/0.525 | — |

Table 8: Engine-versus-engine paired differences under the matched contract (case-cluster bootstrap 95% intervals; wins/losses/ties over 40 cases). Every interval includes zero.

| Comparison | $\Delta$ | 95% CI | W/L/T |
|---|---|---|---|
| Delphi − synthesis engine (matched) | $+0.033$ | $[-0.092, +0.158]$ | 7/5/28 |
| Delphi − synthesis engine (native) | $+0.042$ | $[-0.092, +0.183]$ | 6/6/28 |
| Delphi − documentation engine | $-0.008$ | $[-0.117, +0.092]$ | 6/5/29 |
| Documentation engine − synthesis engine (matched) | $+0.042$ | $[-0.083, +0.167]$ | 10/7/23 |
| Synthesis engine matched − native | $+0.008$ | $[-0.050, +0.075]$ | 5/4/31 |

## 6 Documentation: matched contracts, all four arms

Documentation retrieval is where hosted engines are strongest and where output contracts differ most. The synthesis engine returns a fluent answer with citations; the documentation engine and Delphi return raw context. Scored naively, the synthesis engine leads (identifier hit 0.575 against Delphi's 0.400 and the documentation engine's 0.375). That comparison conflates retrieval with the synthesis model's parametric knowledge.

**Matching the contract, properly.** Our first attempt routed the two raw-context engines through a frozen answer-style synthesis stage (`gpt-5.4-mini`, 1,600 output tokens, one system prompt) and compared them with the synthesis engine's *native* pass, which a reviewer correctly identified as only partially matched: the output format was aligned, the synthesis model and prompt were not. The engine's raw-retrieval mode returned no sources during the recorded round, so we could not route its retrieval at the time. Its synthesized responses, however, carry the five retrieved source chunks it grounded on. We reconstructed that context from the recorded responses (five sources per case, mean 2,754 tokens), packed it under the same 8,000-token budget with the same routine as every other arm, and routed it through the same frozen stage, three repeats, no new hosted calls. Table 7 is now a four-arm comparison under one synthesis model, one prompt, one budget.

Three findings. First, on raw retrieved context the synthesis engine is ahead of Delphi: its reconstructed five-source context hits the gold identifier on 0.475 of cases against Delphi's 0.400 ($k$=5) and 0.425 ($k$=20). Second, under the matched contract the four arms finish within 0.05 of one another and every engine-versus-engine interval includes zero (Table 8); the synthesis engine's matched arm (0.583) is within 0.008 of its native pass, so its own synthesis added nothing our frozen stage did not. Third, the metric saturates: the bare model scores 0.492, so roughly 80% of every arm's score is parametric knowledge, and the retrieval-over-floor deltas are +0.13, +0.13, and +0.09. We do not detect a statistically resolved difference between any two engines under this evaluation. That is the supported sentence; we withdraw "parity or better," which an interval of $[-0.09, +0.16]$ does not establish. Establishing non-inferiority would require a predeclared margin and a sample several times this size.

## 7 RQ1: does any of it help an agent?

**Closed-tool seeding (a replication).** ARB's own seed intervention holds a closed-tool agent fixed and varies its initial context. We replicated it with the policy model `gpt-5.4-mini`, tools `list_dir`/`grep`/`read_file`/`submit`, and 40 paired development cases per arm, adding a July-

Table 9: Executable pilot: 62 C0 SWE-bench Verified instances, mini-SWE-agent with `gpt-5.4-mini`, step budget 50, two independent trajectories per instance and condition. Resolved is the official harness verdict, averaged per instance over the two repeats. * instance-cluster interval excludes zero.

| Condition | Resolved (mean of 2 repeats, of 62) | Rate | Mean cost (USD) | Mean steps |
|---|---|---|---|---|
| No seed | 25.0 | 0.403 | 0.0190 | 8.0 |
| Random files (control) | 28.5 | 0.460 | 0.0251 | 8.3 |
| Delphi top-5 (frozen) | 28.0 | 0.452 | 0.0212 | 7.2 |
| Conventional ladder top-5 | 24.0 | 0.387 | 0.0232 | 7.5 |

*Paired differences in per-instance resolved rate: instance-cluster 95% CI [repository-cluster CI]; instances favouring a/b*

| | |
|---|---|
| Delphi − no seed | $+0.048$ [$-0.008$, $+0.113$] [[$+0.017$, $+0.106$]]; 6/2 |
| Delphi − random | $-0.008$ [$-0.073$, $+0.056$] [[$-0.081$, $+0.022$]]; 6/8 |
| Delphi − conventional | $+0.065$ [$+0.000$, $+0.137$] [[$+0.013$, $+0.147$]]; 12/5 |
| Conventional − no seed | $-0.016$ [$-0.089$, $+0.056$] [[$-0.106$, $+0.062$]]; 5/8 |
| Conventional − random | $-0.073^{*}$ [$-0.145$, $-0.008$] [[$-0.186$, $-0.024$]]; 3/9 |
| Random − no seed | $+0.056$ [$-0.016$, $+0.137$] [[$+0.005$, $+0.179$]]; 9/5 |

Delphi seed. File-F1: no seed $0.046$; random files $0.013$; BM25 seed $0.093$; lexical seed $0.119$; July-Delphi seed $0.234$; oracle $0.858$. The Delphi seed beats no-seed by $+0.188$ $[0.055, 0.294]$ and BM25 by $+0.142$ $[0.059, 0.208]$; against lexical the F1 interval includes zero $[-0.002, 0.190]$. Seeds change which files a frozen agent submits, ordered by seed quality, which is what the benchmark's authors also found. The oracle ceiling shows localization headroom in this harness; it does not show that retrieval dominates reasoning, editing, or validation failures in a repairing agent, and we withdraw that sentence from the earlier draft.

**DS-1000 (unresolved, not negative).** With the frozen generator, 40 cases $\times$ 3 repeats: no retrieval $0.575$; documentation engine $0.567$ ($-0.008$ $[-0.142, 0.125]$); Delphi retrieval+synthesis $0.592$ ($+0.017$ $[-0.125, 0.158]$); synthesis engine $0.642$ ($+0.067$ $[-0.075, 0.200]$). Every interval spans effects from substantial harm to substantial benefit. This experiment does not show that retrieval fails to help; it shows that 40 cases cannot tell.

**Executable pilot (preregistered).** The reviewers' request was direct: measure retrieval and executable outcome on the same tasks. We preregistered a pilot (Appendix C) on the 62 C0 SWE-bench instances whose rankings every system in Table 2 had already produced once. The agent is mini-SWE-agent with `gpt-5.4-mini`, ordinary shell tools in every condition, and a fixed step budget; a seed is a block appended to the issue naming the top five files a recorded ranking produced, with the head of each file, capped at 3,000 tokens. Conditions: no seed; five random non-gold files in the same block (prompting control); Delphi's recorded top five; the conventional ladder's recorded top five. The outcome is verified repair under the official harness, paired by instance, with instance- and repository-cluster intervals. Every condition was run twice (two independent trajectories per instance); Table 9 reports per-instance means over both repeats, and the per-repeat counts are in the text.

The two repeats disagree by more than the effects under study, which is the first result. Resolved counts were 25/30/31/23 (none/random/Delphi/conventional) in the first repeat and 25/27/25/25 in the second: the Delphi seed's six-instance lead over no seed in the first repeat vanished in the second, and the conventional seed's eight-instance deficit shrank to zero. Pooled, no seed effect is resolved. A Delphi seed changes the per-instance repair rate by $+0.048$ $[-0.008, +0.113]$ against no seed, and by $-0.008$ $[-0.073, +0.056]$ against a block of five random non-gold files, which itself sits at $+0.056$ $[-0.016, +0.137]$ over no seed. Whatever the block contributes at this size, it is not separable from the value of naming the right files. The conventional ladder's seed, whose top five named a gold file on 42 instances against Delphi's 47, is the only arm that trails a comparator with an interval excluding zero ($-0.073$ $[-0.145, -0.008]$ against the random control); we do not have an explanation and offer none. Budgets were not binding: agents used 7–8 of 50 steps and cost about two cents per instance; the seed's tokens raised cost by roughly 10–30%, and Delphi's seed reduced steps slightly relative to the random block. The pilot's width is its finding: at $n$=62 with two trajectories, effects of $\pm 0.05$ in verified repair are inside sampling noise, so the study that could answer the title question needs several hundred instances and more repeats, which the per-run cost makes affordable.

## 8 The hosted repository arm

The synthesis engine also markets a repository surface. Scoring it requires 68 exact `(repo, base_commit)` snapshots in the provider's account for the development split alone. Whole-snapshot ingestion of large repositories stalled in a non-terminal `syncing` state; 32/68 pairs remain indexed from an earlier sharded round; a minimal one-shard probe failed to leave `syncing` across a 40-minute deadline and a three-hour watch in August, and a fresh one-shard probe issued on 2026-09-09 with a newly provided credential (which resolves to the same account) timed out in the same state after 40 minutes; the account inventory still shows the August probes syncing. No repository score is reported for that engine in either direction. This is a gap in the evidence, not a result, and after the matched-ladder findings it is no longer the most important gap.

## 9 What we claim, and what we do not

**Claimed.** (1) Delphi leads whole-file BM25 and the official lexical ranker on every set, with intervals excluding zero on the C0 sets for Recall@20 (independent) and for MRR and BCY (SWE-bench, 160 pooled); its Recall@20 lead over the lexical ranker on SWE-bench is a tie. (2) Against a conventional dense+BM25 hybrid built from Delphi's own embedding model with Delphi's own rerankers and expansion attached, Delphi leads by $+0.06$–$0.08$ on Recall@5, Recall@20, and BCY over 160 pooled SWE-bench instances (intervals excluding zero; MRR tied), and trails on the 18-case independent final (two intervals excluding zero). The reranking head accounts for most of the margin over lexical systems on every set. (3) Under one frozen synthesis stage, no two documentation engines are statistically distinguishable at $n$=40, and all sit $0.08$–$0.13$ above a no-retrieval floor. (4) Delphi's retrieved text is repeatable in-process and across restarts with a warm cache; on retrieved-set stability the hosted synthesis engine is at least as stable. (5) Retrieval seeds change a closed-tool agent's file selection, ordered by seed quality, replicating ARB.

**Not claimed.** (1) State of the art, anywhere. (2) Any confirmatory result on the ARB round-3 partition, which is C2. (3) Documentation parity or non-inferiority. (4) Downstream utility on DS-1000 at $n$=40, in either direction. (5) That Delphi's structural branches, tuned weights, or quoted-path demotion add value in general: they separate from the conventional ladder on issue-style queries and not on commit-to-files queries, and the mechanism is untested.

## 10 Conclusion

Measured against baselines built from its own parts, and on cases its development could not see, most of Delphi's advantage is the part any careful practitioner would assemble: an embedding model, BM25, reciprocal-rank fusion, a small cross-encoder, a listwise reranker, and query expansion. What is Delphi's own is worth a further $+0.06$ to $+0.08$ on yield metrics for issue-style queries at $n$=160, and nothing on commit-to-files queries at $n$=18. The claims we made from the re-partitioned ARB set did not survive the ledger that classified how that set had been used, and the claims we made against hosted engines did not survive matching the quantity being measured. What survives is a protocol—exposure classes at the case level, component-matched baselines, matched output contracts, like-for-like determinism, artifact-generated tables—and one open question the pilot begins to answer: whether any of this changes what an agent repairs.

### AI use statement

Generative AI tools were used to help formulate experimental questions, write and test evaluation and product code, operate experiments, inspect failures, search for related work, and draft and edit this manuscript. They were not used to generate benchmark gold labels. Empirical claims are checked against persisted run artifacts and canonical repository snapshots; every table in this paper is generated from those artifacts by a script in the released archive. All AI-assisted work remains subject to human review before submission, and the authors take responsibility for the final content, claims, code, and artifacts.

### Ethics statement

The study uses public open-source repositories and public benchmark records. Repository contents are processed locally except where a named hosted retrieval or model API is itself the evaluated system. We retain commit identifiers and public issue or review text for provenance, but do not intentionally collect private repository data or credentials. Because benchmark results can be used in product marketing, we predeclare the claim rule, preserve failed runs, publish the exposure ledger, and give losses the same prominence as wins.

### Reproducibility statement

The archive accompanying this paper contains the split manifests and case-level exposure ledger, every per-case artifact (queries, gold paths, ranked outputs, synthesized answers, latencies), every paired-analysis JSON, the harness code for every system and comparator including the matched ladder, the frozen serving configuration and the scripts that launch it, the pilot protocol and trajectories, and the script that regenerates every table in this paper from those artifacts. Repository corpora are re-cloned from public GitHub by a provided script; hosted-engine runs require the reader's own credentials.

## References

Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. Locagent: Graph-guided llm agents for code localization. In *ACL*, pp. 8697–8727, 2025. doi: 10.18653/v1/2025.acl-long.426.

Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *ACL*, 2023.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *EMNLP*, 2020.

Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *SIGIR*, 2020.

Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. Contextbench: A benchmark for context retrieval in coding agents. *arXiv preprint arXiv:2602.05892*, 2026.

Bowen Qin and Yi Xie. Agent retrieval bench: Evaluating repository context retrieval for coding agents. *arXiv preprint arXiv:2607.24882*, 2026.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*, 2024.

Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. Repocoder: Repository-level code completion through iterative retrieval and generation. In *EMNLP*, pp. 2471–2484, 2023. doi: 10.18653/v1/2023.emnlp-main.151.

Fuwei Zhang, Yanzhao Zhang, Mingxin Li, Dingkun Long, Lexiang Hu, Pengjun Xie, Zhao Zhang, and Fuzhen Zhuang. Core-bench: A comprehensive benchmark for code retrieval in the era of agentic coding. *arXiv preprint arXiv:2606.11864*, 2026a.

Shaoqiu Zhang, Yuhang Wang, Jialiang Liang, Yuling Shi, Wenhao Zeng, Maoquan Wang, Shilin He, Ningyuan Xu, Siyu Ye, Kai Cai, and Xiaodong Gu. Swe-explore: Benchmarking how coding agents explore repositories. *arXiv preprint arXiv:2606.07297*, 2026b.

Sheng Zhang, Yifan Ding, Shuquan Lian, Shun Song, and Hui Li. Coderag: Finding relevant and necessary knowledge for retrieval-augmented repository-level code completion. In *EMNLP*, pp. 23278–23288, 2025. doi: 10.18653/v1/2025.emnlp-main.1187.

## A Frozen system specification

Frozen commit `91d76c1` of the Delphi repository (branch `feature/generated-source-freeze`), served natively against PostgreSQL with pgvector. Every scored response carried the following attested configuration: embedding model `text-embedding-3-small`; vector mode exact scan (HNSW disabled for scoring; `ef_search` 100 when enabled); hybrid candidate branches vector, BM25, exact symbol, exact path, path affinity, and trigram with reciprocal-rank fusion weights $0.50/0.25/0.20/0.15/0.15/0.05$ and $k{=}60$; 50 fused candidates; cross-encoder `cross-encoder/ms-marco-MiniLM-L-6-v2` over the top 30 with blend $\alpha{=}0.4$ (code-aware reranker disabled); quoted-path demotion when the query envelope carries evidence keys; listwise reranking of the top 20 by `gpt-4o` at temperature 0, seed 1042, 280-character excerpts, 300-token replies; hypothetical-document expansion by `gpt-4o-mini` (temperature 0, seed 1042, 220-token replies) gated on prose-like queries and feeding only the vector branch; persistent SQLite stage cache; API top-$k{=}20$; file-diverse BM25 and path-token branches disabled; agent quality mode with tests, docs, examples, configs, and manifests indexed and generated source retained under 500,000-byte and NUL-content guards. The listwise and expansion system prompts are the strings in `synsc/services/listwise_rerank.py` and `synsc/services/query_expansion.py` at the frozen commit; the matched ladder imports them from the same files. Context packing for BCY@8k uses the official ARB `pack_files` at an 8,000-token budget.

## B Exposure ledger

| Set | Cases | Class | Supports |
|---|---|---|---|
| ARB round-3 partition (positive workflows) | 220 | C2 | validation only |
| ARB round-3 development | 75 | C1 | development |
| Independent commit-to-files final | 18 | C0 | confirmatory |
| Independent commit-to-files development | 48 | C1 | development |
| SWE-bench Verified, round 3 | 62 | C0 | confirmatory |
| SWE-bench Verified, round-4 expansion | 98 | C0 | confirmatory |
| DS-1000 documentation subset | 40 | C1 | development |

Decisions and the evidence that informed them: query-time embedding alignment, RRF fusion, the path-affinity branch, the cross-encoder blend, expansion, and listwise reranking were accepted on round-2 development (75) and round-2 held-out (220) aggregates over the ARB v2 pool that round 3 re-partitioned; quoted-path demotion on July per-workflow rates over all 427 cases; exact scan, single-flight, the persistent cache, and total SQL ordering on round-3 development and determinism probes; generated-source indexing on the gold-path preflight over the independent and ARB development sets; the File-Okapi rejection on both development sets. The machine-readable ledger lists every case ID per set.

## C Executable pilot protocol

Tasks: the 62 round-3 SWE-bench Verified instances. Agent: mini-SWE-agent 2.4.6, text-based action format, `gpt-5.4-mini`, official `x86_64` instance images. Conditions: `none`; `random` (five candidate files drawn with seed 1042 per instance, gold excluded); `delphi` (top five of the frozen stack's recorded ranking); `hybrid_rerank_expand` (top five of the conventional ladder's recorded ranking). Seed block: identical wording, up to five paths with the first 40 lines of each file at the base commit, capped at 3,000 tokens. Budgets: primary step limit 50 with a 2.0 USD cost cap (two independent trajectories per instance and condition were run); a tighter secondary budget was not run because agents used 7–8 steps of 50. Outcome: verified repair under `swebench` 5.0.2 against `SWE-bench/SWE-bench_Verified`; secondary outcomes API cost, steps, whether a gold file was opened, and submission rate. Analysis: paired by instance; instance-cluster and repository-cluster bootstrap $95\%$ intervals (20,000 resamples, seed 1042); exact McNemar on discordant pairs. An interval including zero at $n{=}62$ is reported as unresolved.